

**第一部分：项目全流程概览**

**1.1 核心工作流架构**

深度学习项目遵循系统化开发流程，确保可重复性和高效迭代：

数据采集 → 预处理 → 标注 → 数据集管理 → 模型训练 → 评估 → 部署

**1.2 环境配置最佳实践**

**GPU 环境配置要点：**

- 1. **CUDA 版本匹配：** 显卡驱动 CUDA 版本 ≥ PyTorch 所需 CUDA 版本
- 2. **虚拟环境隔离：** 为每个项目创建独立环境
- 3. **版本一致性：** 记录所有依赖包版本，便于复现

**推荐配置命令：**

```
# 创建环境
conda create -n dl_project python=3.8 -y
conda activate dl_project
# 安装 PyTorch (示例，以官网为准)
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118
# 安装 YOLOv8
pip install ultralytics
```

**第二部分：数据工程体系**

**2.1 数据采集标准化流程**

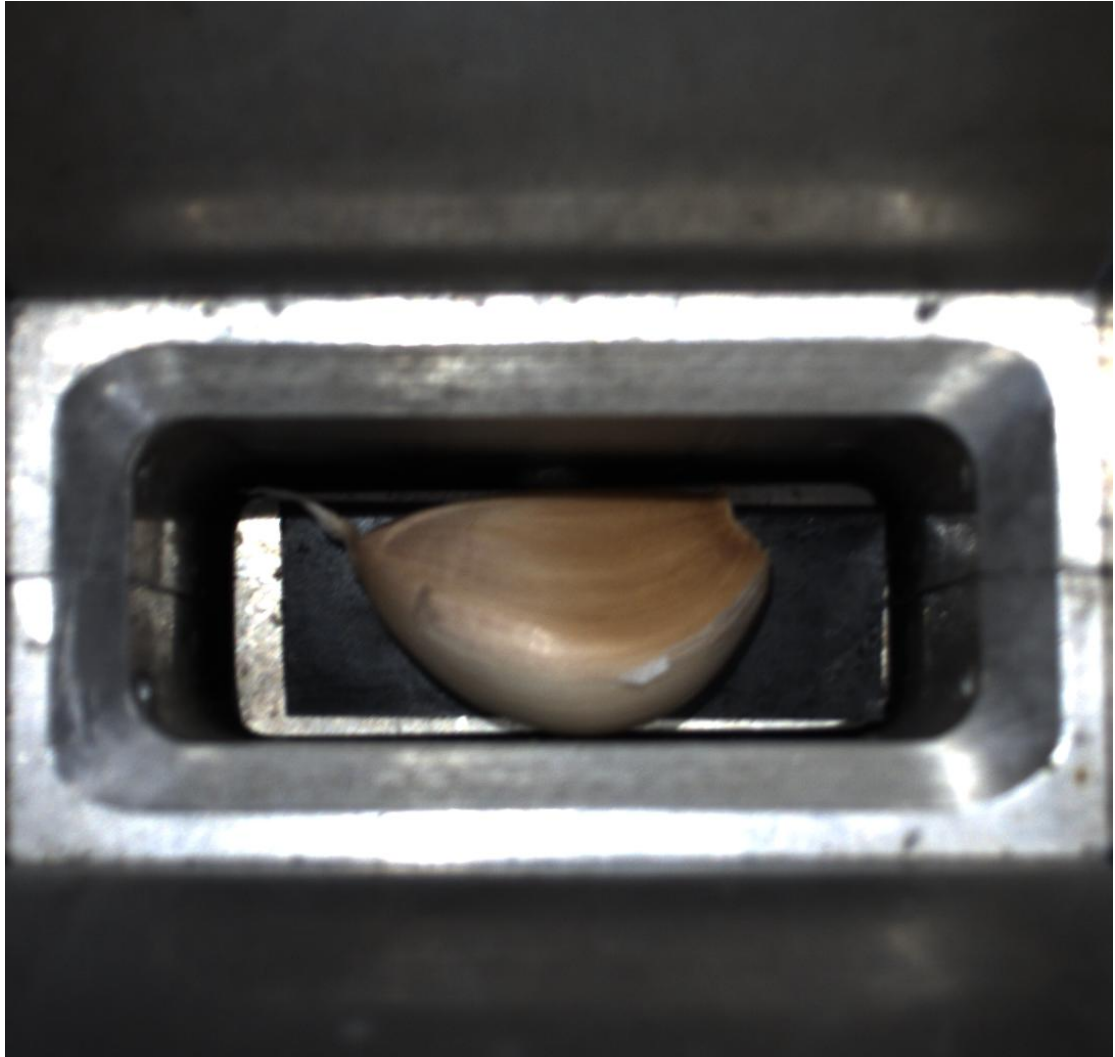
**硬件选型建议：**

- **工业相机：** 全局快门，防果冻效应
- **分辨率：** 越大越好，确保细节捕捉
- **帧率：** 根据应用需求选择（检测快速移动目标需高帧率）

**数据采集策略矩阵：**

| 维度    | 具体方法           | 目的       |
|-------|----------------|----------|
| 视角多样性 | 俯视、仰视、平视       | 提高视角鲁棒性  |
| 光照条件  | 强光、弱光、背光、闪光    | 适应不同光照环境 |
| 距离变化  | 近距离特写、中距离、远距离  | 适应尺度变化   |
| 背景复杂度 | 简单背景、复杂背景、动态背景 | 提高泛化能力   |

例如这是一个大蒜的数据集



编辑

## 2.2 数据标注专业指南

使用 labelme 软件进行标注



编辑

### YOLO 格式深入解析：

类别 ID 中心点 X 归一化 中心点 Y 归一化 宽度归一化 高度归一化

归一化公式详解：

- $x_{box}$  = 框左上角的  $x$  坐标
- $y_{box}$  = 框左上角的  $y$  坐标
- $w_{box}$  = 框的宽度
- $h_{box}$  = 框的高度

归一化的核心优势：

1. **尺度不变性**：模型学习的是相对位置关系
2. **分辨率灵活性**：同一模型可处理不同分辨率输入
3. **数值稳定性**：所有坐标值在 $[0,1]$ 范围内
4. 在 labelme 中的标注 1.检查标签格式类型、数量与图片数量，一般 txt 标签用 labimg 标完后，会有个 classes.txt 文件，要删掉，其次，要开启按文件大小排列，检查空的

The image shows the Labelme software interface. The central canvas displays a photograph of a metallic rectangular frame containing a light-colored, bowl-shaped object. A red bounding box is drawn around the object. The top toolbar includes icons for opening, saving, and editing images. The right sidebar shows the 'Labels' panel with a list of labels, including 'D1mechat\_F1me1w1d\_p10d1d1', and the 'Annotations' panel showing a list of annotations with their bounding boxes.

标注后拿到的 json 文件

```
{
  "version": "5.8.1",
  "flags": {},
  "shapes": [
    {
      "label": "left",
      "points": [
        [
          286.6153846153846,
          473.8076923076923
        ],
        [
          754.3076923076924,
          720.7307692307693
        ]
      ],
      "group_id": null,
      "description": "",
      "shape_type": "rectangle",
      "flags": {},
      "mask": null
    }
  ],
  "imagePath": "00.bmp",
}
```

编辑

关于数据集操作的 python 代码：

DeepStudy C:\Users\30859\Desktop\DeepStudy

01\_data\_preprocessing

01\_video\_frame\_extraction.py  
02\_dataset\_split.py  
03\_data\_augmentation.py  
04\_background\_separation.py  
05\_armor\_extraction.py  
06\_split\_train\_val.py  
07\_convert\_bmp\_to\_jpg.py  
08\_rename\_images\_custom.py  
09\_rename\_txt\_custom.py  
10\_video\_to\_frames\_folder.py  
11\_video\_to\_frames\_fu.py  
12\_video\_to\_frames\_fu\_R.py

02\_label\_processing

01\_txt\_to\_xml.py  
02\_xml\_to\_txt.py  
03\_modify\_class\_id.py  
04\_modify\_json\_labels.py  
05\_yywh\_to\_yolo.py  
06\_generate\_full\_image\_annotation.py  
07\_json\_to\_yolo8\_pose.py  
08\_json\_to\_yolo\_det.py  
12\_rename\_json\_files.py  
13\_txt\_to\_xml\_custom.py  
14\_txt\_to\_xml\_custom\_v2.py  
15\_rename\_xml\_files.py  
16\_json\_to\_txt\_garlic.py  
17\_json\_to\_txt\_garlic\_1.py  
18\_json\_txt\_fu20262.15.py  
19\_json\_txt\_garlic\_copy.py  
20\_json\_txt\_R.py  
21\_json\_txt\_R\_extra.py  
22\_txt\_xml\_garlic.py  
README.md

03\_dataset\_management

01\_match\_check.py  
02\_delete\_txt.py  
03\_rename\_images.py  
04\_rename\_dataset.py  
05\_batch\_rename.py  
06\_extract\_specific\_class.py  
07\_count\_boxes.py  
08\_rename\_images\_v2.py  
09\_rename\_images\_simple.py  
10\_rename\_images\_safe.py

04\_yolo\_training

01\_train\_yolo\_model.py  
02\_evaluate\_pt\_model.py  
03\_compare\_models.py  
04\_adjust\_threshold.py  
05\_check\_pt\_model.py  
06\_check\_pt\_model\_plus.py  
07\_check\_yolo8\_pt.py  
08\_check\_yolo8\_official.py

05\_model\_conversion

01\_pt\_to\_onnx\_openvino.py  
02\_pt\_to\_openvino.py  
03\_onnx\_to\_pt.py  
04\_check\_onnx\_type.py  
05\_get\_onnx\_to\_info.py  
06\_check\_onnx\_type\_v1.py  
07\_check\_onnx\_type\_v2.py  
08\_get\_onnx\_type\_plus.py  
09\_check\_onnx\_health.py  
10\_yolo5\_export.py  
11\_check\_yolo8\_onnx.py  
12\_mainCam\_pt\_to\_onnx.py

06\_model\_inference

01\_onnx\_video\_detection.py  
01\_onnx\_video\_detection\_verified.py  
02\_onnx\_detection\_built.py  
03\_onnx\_detection\_r\_best.py  
04\_onnx\_detection\_v3.py  
05\_onnx\_image\_detection.py  
10\_yolo8\_image\_check.py  
11\_yolo8\_video\_check.py  
12\_yolo5\_test\_line.py  
13\_yolo5\_test\_v1.py  
14\_yolo5\_test\_v2.py  
15\_yolo5\_test\_v3.py  
16\_yolo5\_test\_v4.py  
17\_yolo5\_test\_v5.py  
18\_yolo5\_test\_v6.py  
18\_yolo5\_test\_v7.py

07\_classifier\_training

01\_mlp\_training.py  
01\_mlp\_training.py  
02\_mlp\_to\_onnx.py  
03\_leNet\_training.py  
04\_leNet\_to\_onnx.py  
05\_extract\_bag.py  
06\_process\_cifar100.py  
07\_test\_transform.py  
mlp.onnx

README.md

rm\_vision.svg

08\_utils

01\_yolo5\_to\_v8\_split.py  
02\_generate\_yolo5\_train\_val.py  
03\_stations.py  
04\_check\_code.py  
05\_video\_screenshot.py  
06\_compare\_onnx\_models.py  
09\_image\_grayscale.py  
10\_onnx\_export\_utils.py  
11\_data\_augmentation\_flip.py  
12\_dataset\_split\_manual.py  
13\_random\_name\_picker.py  
14\_fu\_label\_classes.py  
15\_generate\_file\_list.py  
16\_custom\_to\_voc\_json.py  
17\_create\_voc\_main\_data.py  
18\_extract\_keypoints.py

09\_documentation

environment.yml  
README.md  
数字孪生模型参考源码.zip

外部资源

临时文件和控制台

编辑

## 2.3 数据增强策略库

### 自动增强 (YOLO 内置):

- **Mosaic 增强**: 四图拼接, 提升小目标检测
- **MixUp**: 图像混合, 提升鲁棒性
- **HSV 调整**: 模拟不同光照条件
- **几何变换**: 旋转、缩放、裁剪

### 手动增强 (特殊需求):

# 自定义增强示例

```
import albumentations as A
transform = A.Compose([
    A.RandomRotate90(p=0.5),
    A.RandomBrightnessContrast(p=0.2),
    A.GaussNoise(p=0.3),
])
```

## 2.4 ROI (感兴趣区域) 技术详解

### ROI 概念与价值:

- **定义**: 在图像中划定特定区域, 仅对该区域进行处理
- **核心价值**:
  - **计算效率**: 减少处理像素数量
  - **精度提升**: 避免背景干扰
  - **资源优化**: 在有限硬件上实现复杂任务

### ROI 应用模式:

#### 1. 两级检测架构:

第一级: YOLO 检测 → 获取 ROI 坐标

第二级: ROI 裁剪 → 分类器识别

#### 1. 代码实现示例:

```
def extract_roi(image, bbox):
    """根据 bbox 裁剪 ROI 区域"""
    x1, y1, x2, y2 = bbox
    roi = image[y1:y2, x1:x2]
    return roi

# 应用场景: 装甲板数字识别
detections = yolo_model(image) # 一级检测
for det in detections:
    if det.class_id == "armor":
        roi = extract_roi(image, det.bbox)
        number = classifier(roi) # 二级分类
```

## 第三部分: 模型架构与原理

### 3.1 卷积神经网络 (CNN) 核心组件

#### 卷积层 (Convolutional Layer):

- **核心作用**: 特征提取, 通过滑动窗口学习局部模式
- **关键参数**:
  - **卷积核**: 3×3、5×5 等, 决定感受野大小
  - **步长 (Stride)**: 决定滑动步幅, 影响输出尺寸

- **填充 (Padding)**: 保持输入输出尺寸一致
- **通道数**: 决定提取特征的种类数量

#### 批量归一化 (Batch Normalization):

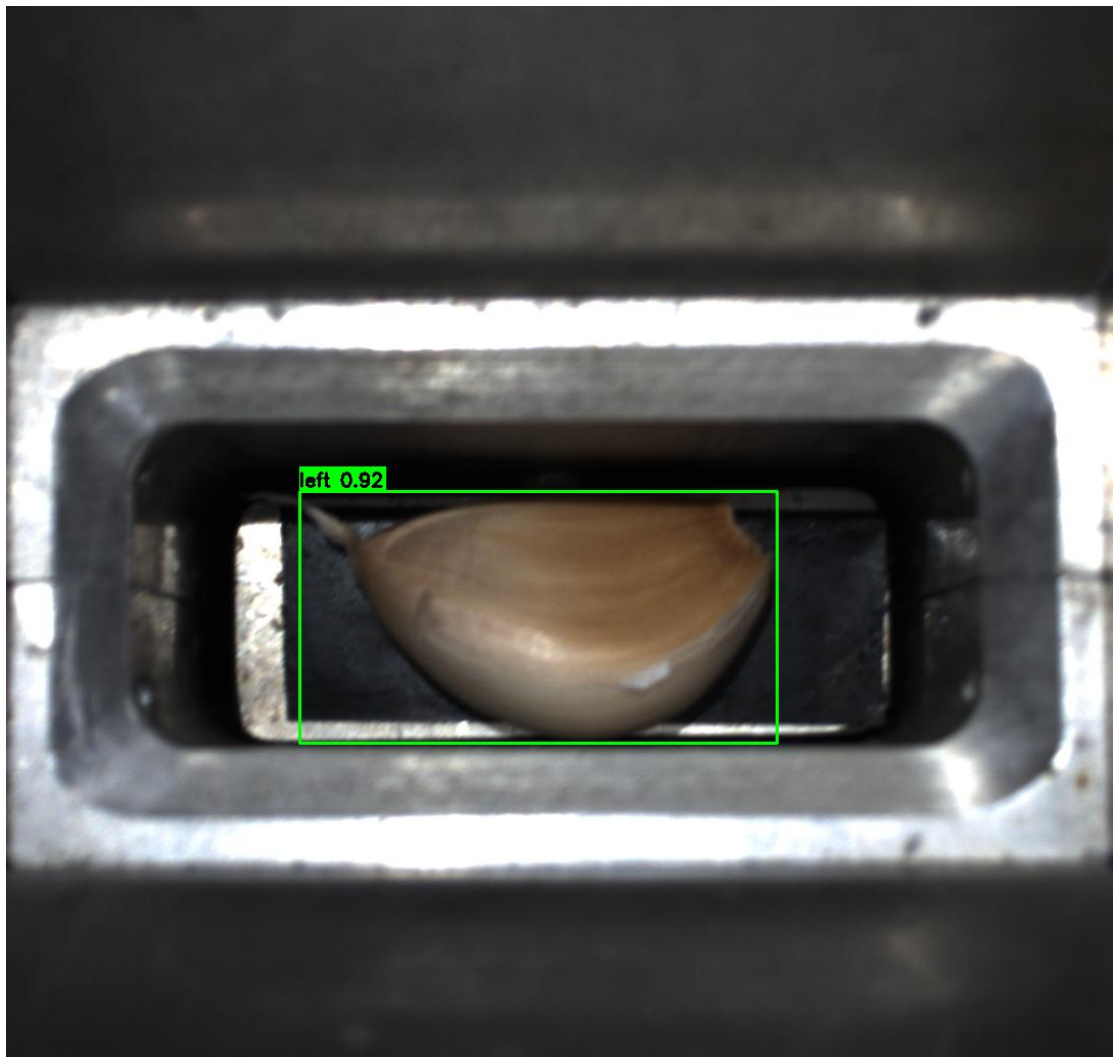
- **问题背景**: Internal Covariate Shift (内部协变量偏移)
- **解决方案**: 对每层输入进行标准化
- **核心优势**:
  - 加速训练收敛
  - 允许使用更大学习率
  - 提供轻微正则化效果

#### ResNet (残差网络):

- **核心创新**: 跳跃连接 (Skip Connection)
- **残差块结构**:  $y = F(x) + x$
- **解决的核心问题**: 深度网络梯度消失
- **YOLOv8 中的应用**: C2f 模块借鉴了残差思想

### 3.2 视觉任务全家桶

#### 最常见的是目标检测 (Object Detection):



#### 编辑

- **任务**: 定位 (框位置) + 分类 (是什么)

- 代表算法：YOLO 系列、SSD、R-CNN 系列

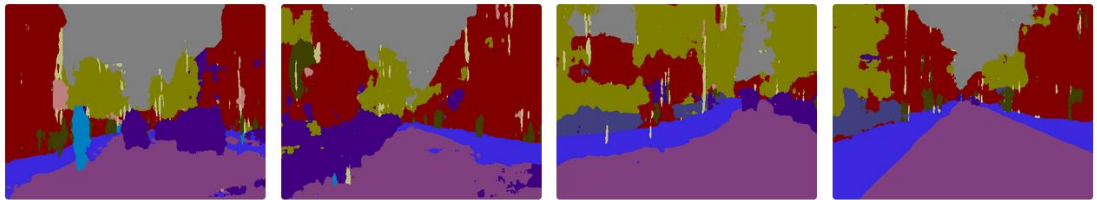
然后还有语义分割（Semantic Segmentation）：

- 任务：像素级分类，不区分个体
- 输入：图像 → 输出：相同尺寸的分类图
- 应用场景：
  - 道路分割（区分路面、人行道、车辆）
  - 医疗影像（区分正常组织与病变区域）
- 与实例分割的区别：

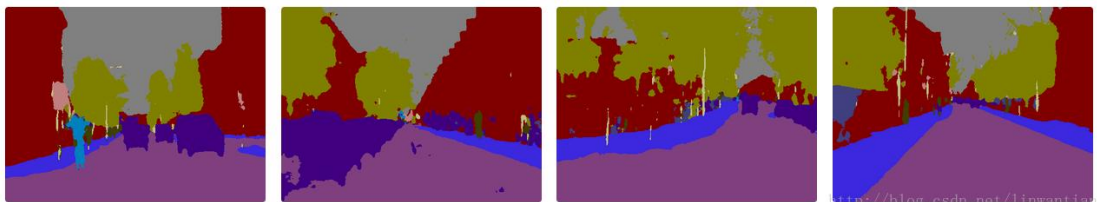
Input Image



SegNet-Basic Segmentation



SegNet Segmentation



<https://blog.csdn.net/liwentian>

- 

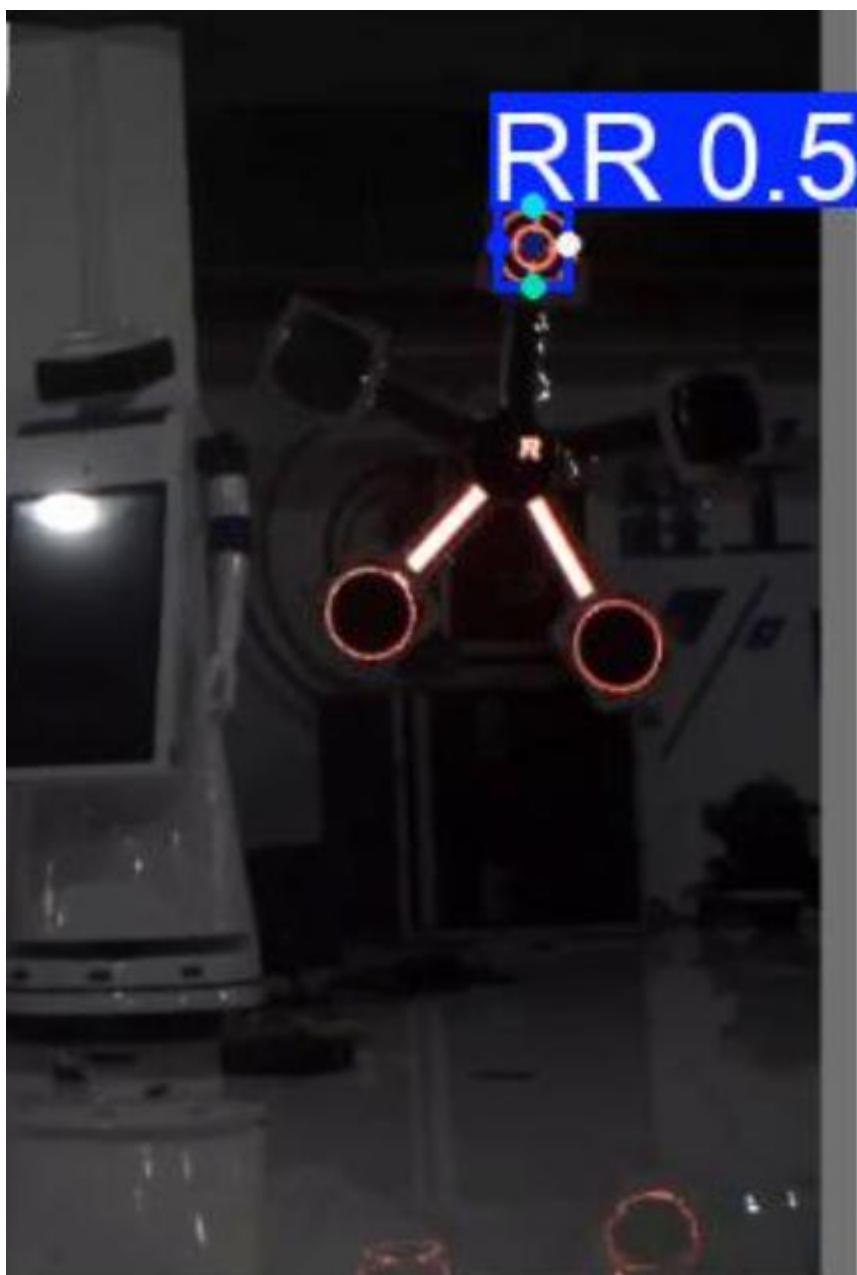
编辑

语义分割：所有"车"像素标记为同一类别

实例分割：每辆车标记为不同实例

姿态估计（Pose Estimation）：





编辑

- 任务：检测关键点并连接成骨架
- YOLOv8-pose：检测+姿态一体化
- 应用：能量机关 R 标定位、人体姿态分析

### 3.3 目标检测算法演进

#### 两阶段检测器（R-CNN 系列）：

- 流程：区域提议 → 特征提取 → 分类回归
- 代表：R-CNN、Fast R-CNN、Faster R-CNN
- 特点：精度高，速度慢

#### 单阶段检测器（SSD/YOLO）：

- 流程：直接预测边界框和类别
- SSD (Single Shot MultiBox Detector)：
  - 多尺度特征图预测
  - 默认框 (Default Boxes) 机制

- **YOLO (You Only Look Once):**
  - 将检测视为回归问题
  - 网格化预测，速度快

#### YOLO 进化关键点:

- **v1-v3:** 锚框 (Anchor) 机制
- **v4:** 大量优化技巧集成
- **v5:** 工程化改进，易于部署
- **v8:** 锚框免费 (Anchor-Free)，解耦头
- **v11:** 微观算子打磨，多任务大一统
- **v26:** 剔除历史包袱，端到端架构落地

### 3.4 模型选择策略

#### 根据硬件选择模型:

| 硬件平台         | 推荐模型      | 推理速度      | 精度 |
|--------------|-----------|-----------|----|
| 树莓派 4B       | YOLOv8n   | 15-20 FPS | 中等 |
| Jetson Nano  | YOLOv8s   | 30-40 FPS | 良好 |
| RTX 3060 笔记本 | YOLOv8m   | 60+ FPS   | 优秀 |
| RTX 4090 服务器 | YOLOv8l/x | 100+ FPS  | 顶尖 |

#### 根据任务复杂度选择:

- **简单场景:** 少量类别，背景干净 → YOLOv8n/s
- **复杂场景:** 多类别，遮挡严重 → YOLOv8m/l
- **高精度要求:** 小目标检测，密集场景 → YOLOv8l/x

### 3.5 面向边缘的模型联合优化策略

对于机器人、无人机等边缘计算场景，模型必须在有限的计算资源、内存和功耗下实现高效推理。单一技术往往难以兼顾，联合优化成为关键。

- **内存-算力协同优化框架:** 结合**量化感知训练**和**结构化模型剪枝**，可以实现远超单一技术的优化效果。
- **量化感知训练:** 在训练中模拟低精度 (如 INT8) 计算，让模型提前“适应”量化误差，精度损失通常小于 1%。
- **结构化剪枝:** 移除网络中不重要的通道或层，直接减少参数和计算量。
- **联合效益:** 实验表明，通过联合优化可将模型体积和计算量压缩 **90%** 以上，推理延迟降低 **10 倍**，同时将精度损失控制在 2% 以内。

## 第四部分：训练与优化

### 4.1 训练流程科学化

#### 完整训练阶段:

1. **热身阶段:** 3 个 epoch，学习率从 0 线性上升
2. **主训练阶段:** 模型参数更新，损失下降
3. **微调阶段:** 最后 10 个 epoch，关闭马赛克增强

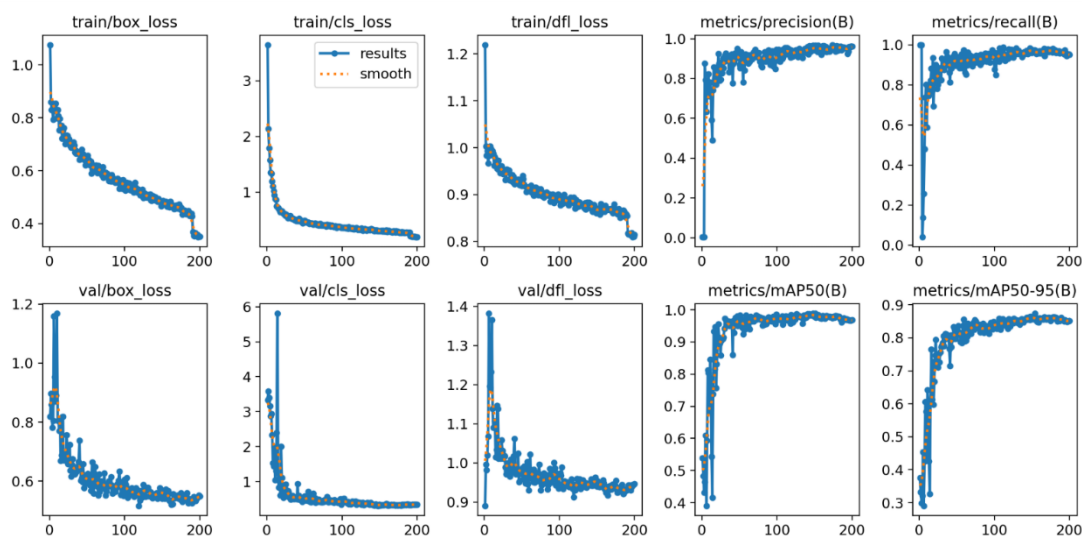
#### 关键超参数设置:

| 参数         | 建议值       | 说明        |
|------------|-----------|-----------|
| batch_size | 16/32/64  | 显存允许下越大越好 |
| workers    | CPU 核心数/2 | 数据加载线程数   |
| patience   | 50-100    | 早停耐心值     |

optimizer SGD/AdamW 大数据集 SGD, 小数据集 AdamW

cos\_lr TRUE 余弦退火学习率

## 4.2 过拟合与欠拟合诊断与解决



编辑

### 过拟合 (Overfitting):

- 现象: 训练损失低, 验证损失高
- 根本原因: 模型复杂度过高, 记忆训练数据
- 诊断指标: Train Loss << Val Loss, mAP 差异大
- 解决方案:
  - 增加数据量 (最有效)
  - 数据增强 (Mosaic、MixUp、CutMix)
  - 正则化 (权重衰减、Dropout)
  - 早停 (Early Stopping)
  - 简化模型 (使用更小模型)

### 欠拟合 (Underfitting):

- 现象: 训练损失和验证损失都高
- 根本原因: 模型能力不足或训练不充分
- 诊断指标: Loss 下降缓慢或停滞
- 解决方案:
  - 增加模型复杂度 (使用更大模型)
  - 延长训练时间 (增加 epoch)
  - 提高学习率
  - 检查数据质量 (标签是否正确)
  - 减少正则化强度

## 4.3 SoftMax 回归详解

### 特性与优势:

1. 概率解释: 输出可直接视为分类概率
2. 可微性: 便于梯度计算和反向传播
3. 放大差异: 高分更高, 低分更低

- 4. 多分类标准：几乎用于所有多分类任务

YOLO 中的应用：

- 分类分支：每个锚点预测各类别概率
- 置信度计算：物体置信度 × 类别概率

## 第五部分：序列模型与语言处理

### 5.1 从视觉到序列的拓展

RNN（循环神经网络）：

- 核心思想：引入时间维度，具有记忆功能
- 局限性：梯度消失/爆炸，长程依赖问题

LSTM（长短期记忆网络）：

- 关键创新：门控机制
  - 遗忘门：决定丢弃哪些信息
  - 输入门：决定存储哪些新信息
  - 输出门：决定输出什么信息
- 细胞状态：信息高速公路，梯度可无损传递
- 应用场景：时间序列预测、文本生成

### 5.2 注意力机制革命

核心概念：

- 核心思想：动态权重分配，关注重要信息
- 自注意力公式： $\text{Attention}(Q,K,V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$ 
  - Q：查询（我想要什么）
  - K：键（你有什么）
  - V：值（实际内容）

视觉注意力：

- 空间注意力：关注图像的特定区域
- 通道注意力：关注特征图的特定通道
- 应用：帮助模型关注关键特征，提升小目标检测

### 5.3 语言模型与语音模型

语言模型（Language Model）：

- 核心任务：预测下一个词的概率分布
- 数学表达： $P(w_t | w_1, w_2, \dots, w_{t-1})$
- 现代 LLM：基于 Transformer 的生成式预训练模型
- 应用：文本生成、翻译、代码补全

语音模型（Speech Model）：

- 与语言模型的关系：将音频信号视为“听觉语言”
- 关键技术：
  - 音频编码：将波形转为频谱图或特征向量
  - 序列建模：使用 CNN+Transformer 处理时间序列
  - 解码：转为文本或直接理解
- 应用：语音识别、语音合成、语音情感分析

多模态融合：

- 视觉+语言：图像描述、视觉问答
- 语音+视觉：唇语识别、多模态情感分析
- 发展趋势：统一的多模态大模型

第六部分：评估与部署

6.1 专业评估指标体系

目标检测核心指标：

| 指标        | 公式/定义                           | 适用场景              |
|-----------|---------------------------------|-------------------|
| mAP50     | IoU 阈值 0.5 的平均精度                | 实时检测，如 RoboMaster |
| mAP50-95  | IoU 阈值 0.5-0.95 的平均             | 高精度要求，如医疗         |
| Precision | $TP/(TP+FP)$                    | 强调误报率低的场景         |
| Recall    | $TP/(TP+FN)$                    | 强调漏检率低的场景         |
| F1-Score  | $2 \times P \times R / (P + R)$ | 平衡精确率和召回率         |

混淆矩阵分析：

- **对角线**：正确分类的比例
- **非对角线**：混淆情况（A 类误认为 B 类）
- **应用**：发现模型特定弱点，针对性改进

6.2 部署优化策略

模型导出最佳实践：

```
from ultralytics import YOLO
model = YOLO('best.pt')
# 针对不同 Runtime 的导出建议
# 1. 通用 ONNX (适用于 ONNX Runtime, OpenVINO)
model.export(format='onnx', opset=11, simplify=True)
# 2. 针对 TensorRT (注意：通常建议直接用 Ultralytics 的 engine 导出)
# model.export(format='engine', half=True) # FP16 模式，速度快
# 3. 针对 OpenVINO
# model.export(format='openvino')
```

推理优化技术：

| 技术   | 精度损失 | 速度提升     | 适用场景 |
|------|------|----------|------|
| FP32 | 无    | 基准       | 精度优先 |
| FP16 | 极小   | 1.5-2×   | 平衡场景 |
| INT8 | 中等   | 2-3×     | 速度优先 |
| 模型剪枝 | 可控   | 1.5-2×   | 边缘设备 |
| 知识蒸馏 | 小    | 1.2-1.5× | 模型压缩 |

6.3 全链路质量保障

测试策略：

1. **单元测试**：每个脚本功能验证
2. **集成测试**：全流程打通测试
3. **压力测试**：高负载下的稳定性
4. **A/B 测试**：新旧模型对比

监控指标：

- **推理延迟**：端到端处理时间
- **吞吐量**：单位时间处理数量
- **资源使用**：CPU/GPU/内存占用
- **准确率跟踪**：线上数据性能监控

## 6.4 边缘与移动端的深度部署优化

将模型部署到资源受限的设备时，需要在转换、压缩和硬件适配各环节精细优化。

### 1. 模型转换与算子兼容

1. **格式选择**：ONNX 通用性好；TorchScript 在 PyTorch 生态内损耗低。
2. **常见问题**：遇到不支持的算子时，需进行算子替换或图优化（如常量折叠、移除训练专用层）。

硬件感知的性能榨取

- **CPU 优化**：利用 NEON 指令集实现向量化计算，并通过多线程调度平衡负载。
- **GPU 优化**：利用纹理内存和计算着色器优化并行计算。
- **专用 AI 加速器**：针对 NPU 进行算子融合和数据布局转换，能大幅提升能效。

**自动化编译优化**：使用如 Apache TVM 及其 AutoScheduler 等工具，可以自动为不同的边缘硬件搜索最优的算子实现方案，显著提升执行效率。

## 6.5 部署后的系统监控与持续迭代

模型上线并非终点，而是持续运营的开始。

- **性能与异常监控**：需建立**实时看板**监控推理延迟、吞吐量、资源利用率等指标。设置**异常检测机制**，如监控数据分布漂移（例如使用 KL 散度），在指标异常时触发告警。

持续优化闭环：

- **A/B 测试**：通过流量分层采样，科学评估新模型版本在真实场景下的效果。

**自动化评估与调优**：利用工具对智能体工作流进行基准测试，并自动调整关键超参数。

**数据回流与模型更新**：建立管道，将生产环境中的困难样本、新案例回流，用于迭代训练，形成闭环。

## 6.6 主流部署框架与 Runtime 详解

在模型部署环节，ONNX 通常是通用的“中间语言”，而 TensorRT、OpenVINO 等则是针对特定硬件进行极致优化的“终极执行环境”。

### 1. ONNX (Open Neural Network Exchange)

- **定位**：通用的模型中间表示格式。
- **作用**：解决框架壁垒。无论你是用 PyTorch (Ultralytics) 还是 TensorFlow 训练的模型，都可以先导出为 .onnx 格式。它定义了一个标准的计算图，使得模型可以在不同的训练框架和推理引擎之间迁移。
- **YOLO 实践**：model.export(format='onnx')生成的文件可以被几乎所有主流推理框架读取。它是通往 TensorRT 或 OpenVINO 的必经之路。

### 2. TensorRT (NVIDIA)

- **定位**：NVIDIA GPU 专属的高性能推理 Runtime。

核心优势：

- **算子融合 (Layer Fusion)**：将 Conv + BN + ReLU 合并为一个算子，减少显存读写。
- **精度校准 (INT8/FP16)**：利用 NVIDIA 显卡的 Tensor Cores 进行低精度高速计算。
- **显存优化**：为模型分配最优的显存空间。
- **适用场景**：服务器端、Jetson 系列边缘设备（如 Jetson Nano/Orin）。这是目前 YOLO 在 NVIDIA 硬件上跑得最快的方案。

### 3. OpenVINO (Intel)

- **定位**：Intel 硬件专属的推理工具套件。

核心优势：

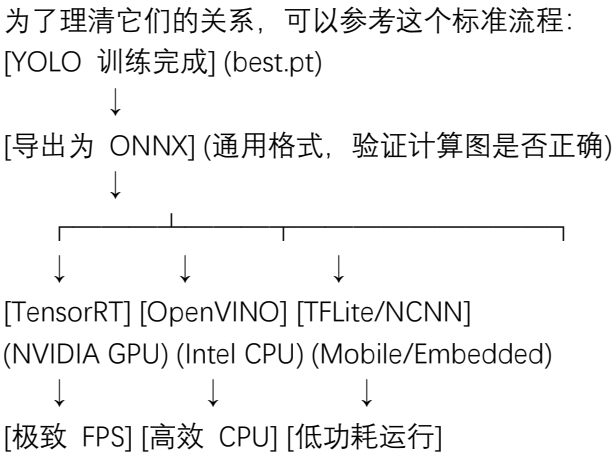
- 针对 Intel CPU、核显（Iris Xe）、以及神经计算棒（NCS2）进行了深度优化。

- 支持通过 ONNX 导入模型，并转换为 .xml 和 .bin 文件。
- **适用场景：**工业电脑（大多使用 Intel CPU）、海康/大华等安防设备、普通笔记本。  
在没有独立显卡的情况下，OpenVINO 能让 YOLO 在 CPU 上跑出惊人的帧率。

4. 其他常见 Runtime

| 框架/Runtime   | 厂商/平台  | 特点与适用场景                                        |
|--------------|--------|------------------------------------------------|
| ONNX Runtime | 微软/开源  | 跨平台通用性强, 支持 CPU/GPU, API 简单易用, 适合快速验证。         |
| TFLite       | Google | 专为 Android 移动端和嵌入式设备设计, 模型体积小, 功耗低。            |
| NCNN         | 腾讯     | 针对手机端 (ARM 架构) 优化的极轻量级推理框架, 国内移动端开发首选。         |
| DeepStream   | NVIDIA | 基于 TensorRT 构建的视频分析框架, 适合多路视频流 (如几十路摄像头) 并发处理。 |

5. 部署链路示意图



第七部分：项目管理与最佳实践

7.1 代码组织结构

text

```
project/
├── data/ # 数据集
│   ├── raw/ # 原始数据
│   ├── processed/ # 处理后数据
│   └── labels/ # 标签文件
├── src/ # 源代码
│   ├── data_processing/ # 数据处理模块
│   ├── models/ # 模型定义
│   ├── training/ # 训练脚本
│   └── inference/ # 推理脚本
├── experiments/ # 实验记录
│   ├── exp001/ # 实验 1
│   └── exp002/ # 实验 2
└── configs/ # 配置文件
```

|—— docs/ # 文档  
|—— tests/ # 测试代码

## 7.2 实验管理

关键记录内容：

1. **环境信息**：Python 版本、库版本、CUDA 版本
2. **超参数**：完整训练配置
3. **数据信息**：数据集大小、分布、增强策略
4. **实验结果**：评估指标、可视化图表
5. **观察发现**：训练过程中的异常或洞见